



Tehnologii WEB

Curs 5

CSS - continuare

Cod Teams: **6gyjd1x**

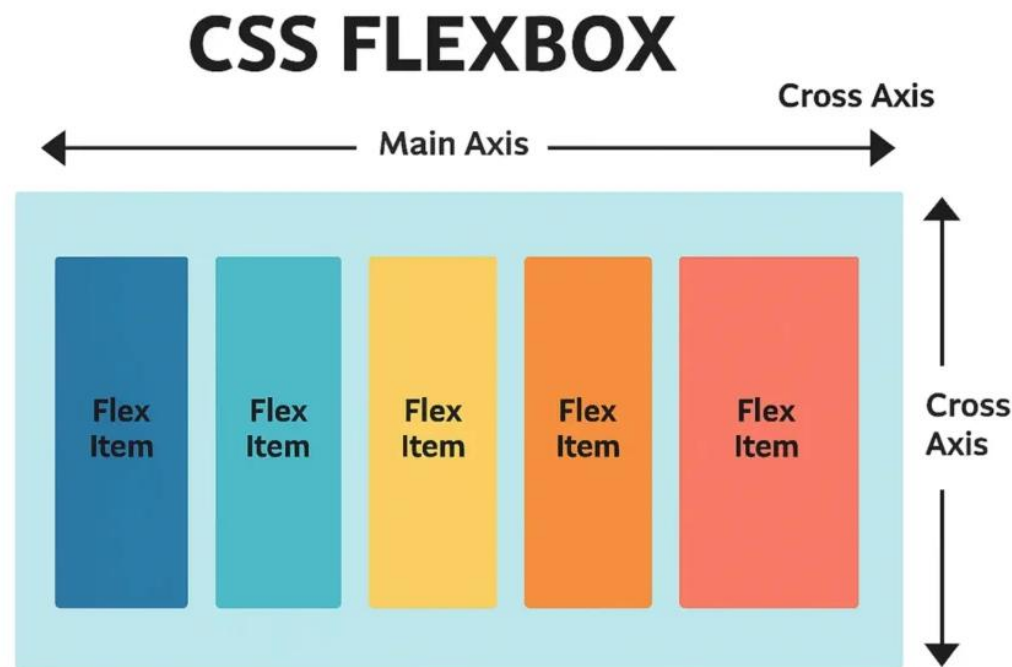
Anul III, semestrul II

2026

CSS Flexbox

Modulul Flexbox Layout (Flexible Box) își propune să ofere o modalitate mai eficientă de a amplasa, alinia și distribui spațiul între elementele dintr-un container, chiar și atunci când dimensiunea lor este necunoscută și/sau dinamică (de unde și cuvântul „flex”).

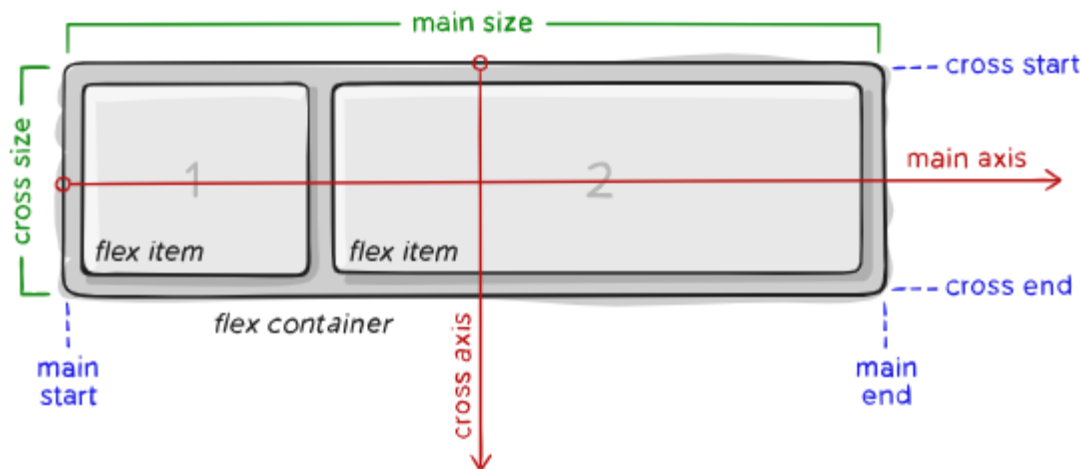
Ideea principală din spatele layout-ului flexibil este de a oferi containerului posibilitatea de a modifica lățimea/înălțimea (și ordinea) elementelor sale pentru a umple cât mai bine spațiul disponibil (în principal pentru a se adapta la toate tipurile de dispozitive de afișare și dimensiuni de ecran). Un container flexibil extinde elementele pentru a umple spațiul liber disponibil sau le micșorează pentru a preveni supraaglomerarea.



Noțiuni de bază și terminologie Flexbox

Întrucât flexbox este un modul întreg și nu o singură proprietate, implică o mulțime de lucruri, inclusiv întregul set de proprietăți. Unele dintre ele sunt destinate a fi setate pe container (elementul părinte, cunoscut sub numele de „container flex”), în timp ce altele sunt destinate a fi setate pe fii (numite „elemente flex”).

Dacă aspectul „regulat” se bazează atât pe direcțiile de curgere în bloc, cât și pe cele în linie, aspectul flexibil se bazează pe „direcțiile de curgere flexibile”.



Elementele vor fi așezate fie urmând axa main axis (de la main-start la main-end), fie după axa transversală (de la cross-start la cross-end).

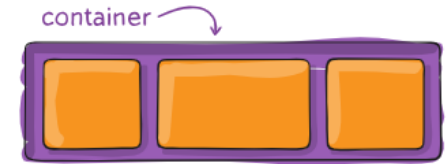
- **axa principală** – Axa principală a unui container flexibil este axa principală de-a lungul căreia sunt așezate elementele flexibile. Atenție, nu este neapărat orizontală; depinde de flex-directionproprietate (vezi mai jos).
- **main-start | main-end** – Elementele flex sunt plasate în container începând de la main-start și mergând spre main-end.
- **dimensiune principală** – Lățimea sau înălțimea unui element flexibil, oricare dintre acestea se află în dimensiunea principală, reprezintă dimensiunea principală a elementului. Proprietatea de dimensiune principală a elementului flexibil este fie proprietatea „lățime”, fie proprietatea „înălțime”, oricare dintre acestea se află în dimensiunea principală.
- **axă transversală** – Axa perpendiculară pe axa principală se numește axă transversală. Direcția sa depinde de direcția axei principale.
- **început încrucișat | capăt încrucișat** – Liniile flexibile sunt umplute cu articole și plasate în container, începând de pe partea de început încrucișat a containerului flexibil și mergând spre partea de capăt încrucișat.
- **dimensiunea încrucișată** – Lățimea sau înălțimea unui element flexibil, oricare dintre ele se află în dimensiunea încrucișată, este dimensiunea încrucișată a elementului. Proprietatea dimensiunii încrucișate este oricare dintre „lățime” sau „înălțime” care se află în dimensiunea încrucișată.

Proprietăți CSS Flexbox

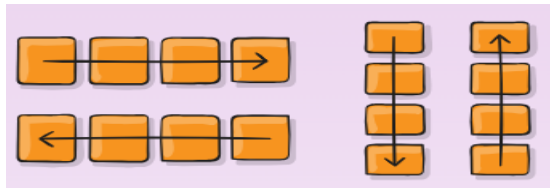
Proprietăți pentru elementul părinte (container flexibil)

Display: Aceasta definește un container flex; inline sau block, în funcție de valoarea dată. Activează un context flex pentru toți copiii săi direcți. Coloanele CSS nu au niciun efect asupra unui container flex.

```
.container { display: flex; /* or inline-flex */ }
```



Flex-direction: Aceasta stabilește axa principală, definind astfel direcția în care elementele flex sunt plasate în containerul flex. Flexbox este (în afară de încadrarea opțională) un concept de layout unidirecțional. Gândiți-vă la elementele flex ca fiind dispuse în principal fie în rânduri orizontale, fie în coloane verticale.



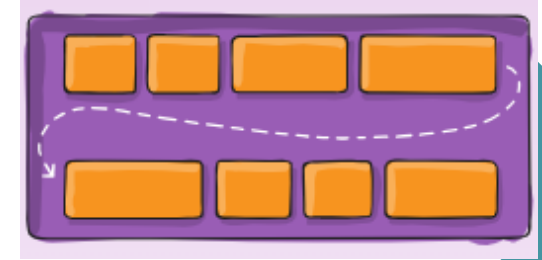
```
.container { flex-direction: row | row-reverse | column |  
column-reverse; }
```

- **row(implicit):** de la stânga la dreapta înăuntru ltr; de la dreapta la stânga înăuntru rtl
- **row-reverse:** de la dreapta la stânga înăuntru ltr; de la stânga la dreapta înăuntru rtl
- **Column** la fel ca, row dar de sus în jos
- **column-reverse** la fel ca, row-reverse dar de jos în sus

Flex-wrap: În mod implicit, toate elementele flexibile vor încerca să se încadreze pe o singură linie. Puteți modifica acest lucru și puteți permite elementelor să se încadreze după cum este necesar cu ajutorul acestei proprietăți.

```
.container { flex-wrap: nowrap | wrap | wrap-reverse; }
```

- **nowrap** (implicit): toate elementele flexibile vor fi pe o singură linie
- **Wrap** - Elementele flexibile se vor încadra pe mai multe linii, de sus în jos.
- **wrap-reverse** - Elementele flexibile se vor încadra pe mai multe linii de jos în sus.



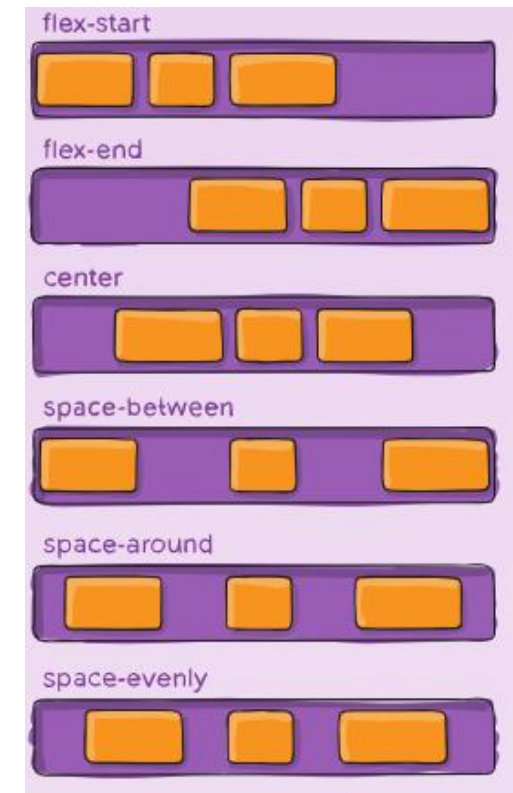
Flex-flow: Aceasta este o prescurtare pentru proprietățile flex-direction și flex-wrap, care împreună definesc axa principală și axa transversală a containerului flex. Valoarea implicită este row nowrap. (flux flexibil)

```
.container { flex-flow: column wrap; }
```

Justify-content: Aceasta definește alinierea de-a lungul axei principale. Ajută la distribuirea spațiului liber suplimentar rămas atunci când toate elementele flexibile de pe o linie sunt inflexibile sau sunt flexibile, dar au atins dimensiunea maximă. De asemenea, exercită un anumit control asupra alinierii elementelor atunci când acestea depășesc linia.

```
.container { justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly | start | end | left | right ... + safe | unsafe; }
```

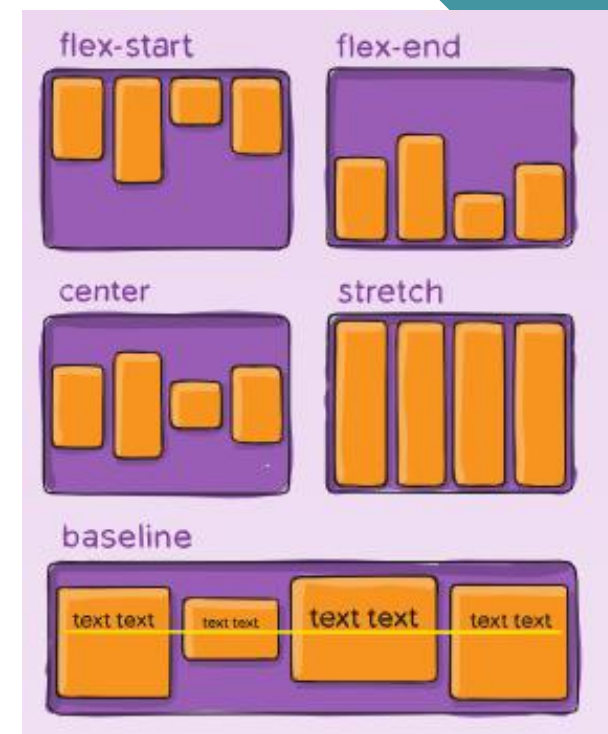
- **flex-start(implicit):** articolele sunt împachetate spre începutul direcției flexibile.
- **flex-end** - Articolele sunt ambalate spre sfârșitul direcției de flexie.
- **start:** articolele sunt ambalate spre începutul direcției writing-mode.
- **end:** articolele sunt ambalate spre sfârșitul direcției writing-mode.
- **left:** elementele sunt împachetate spre marginea stângă a containerului, cu excepția cazului în care acest lucru nu are sens cu flex-direction, atunci se comportă ca start.
- **right:** elementele sunt împachetate spre marginea dreaptă a containerului, cu excepția cazului în care acest lucru nu are sens cu flex-direction, atunci se comportă ca end.
- **center:** elementele sunt centrate de-a lungul liniei
- **space-between** - Elementele sunt distribuite uniform pe linie; primul element este pe linia de început, ultimul element pe linia de sfârșit
- **space-around:** elementele sunt distribuite uniform pe linie, cu spațiu egal în jurul lor. Rețineți că vizual spațiile nu sunt egale, deoarece toate elementele au spațiu egal pe ambele părți. Primul element va avea o unitate de spațiu pe marginea containerului, dar două unități de spațiu între următorul element, deoarece acesta are propria sa spațiere aplicată.
- **space-evenly:** elementele sunt distribuite astfel încât spațierea dintre oricare două elemente (și spațiul până la margini) să fie egală.



Align-items: Aceasta definește comportamentul implicit pentru modul în care elementele flexibile sunt dispuse de-a lungul axei transversale pe linia curentă. Gândiți-vă la aceasta ca la justify-content versiunea pentru axa transversală (perpendiculară pe axa principală).

```
.container { align-items: stretch | flex-start | flex-end | center | baseline | first baseline | last baseline | start | end | self-start | self-end + ... safe | unsafe; }
```

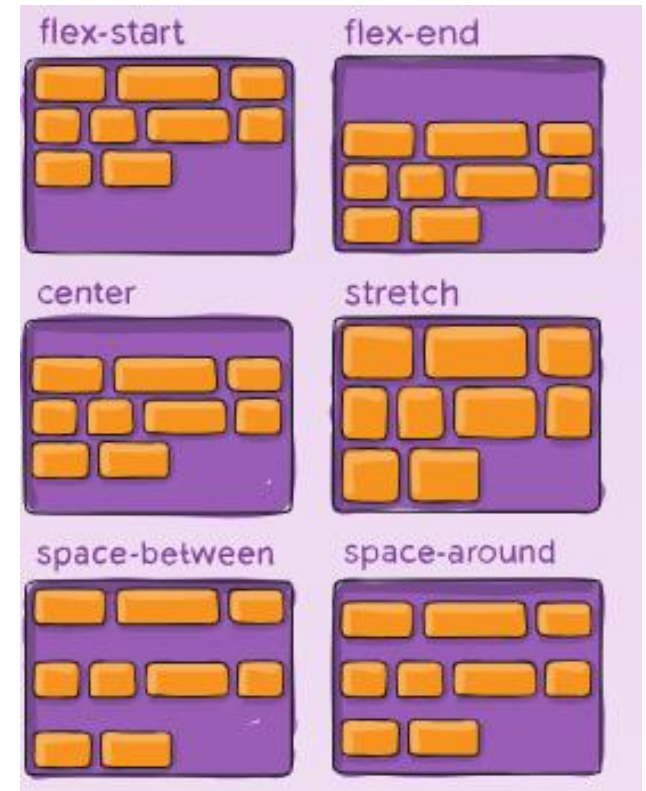
- stretch(implicit): se întinde pentru a umple containerul (respectând în continuare lățimea minimă/lățimea maximă)
- flex-start/ start/ self-start: obiectele sunt plasate la începutul axei transversale. Diferența dintre acestea este subtilă și constă în respectarea flex-direction regulilor sau a writing-mode regulilor.
- flex-end/ end/ self-end: obiectele sunt plasate la capătul axei transversale. Diferența este din nou subtilă și constă în respectarea flex-direction regulilor versus writing-mode regulile obișnuite.
- center: elementele sunt centrate pe axa transversală
- baselineelementele sunt aliniate, cum ar fi liniile lor de bază aliniate



Align-content: Aceasta aliniază liniile unui container flexibil în interiorul acestuia atunci când există spațiu suplimentar pe axa transversală, similar cu modul în care justify-content se aliniază elementele individuale în cadrul axei principale.

```
.container { align-content: flex-start | flex-end | center | space-between |  
space-around | space-evenly | stretch | start | end | baseline | first  
baseline | last baseline + ... safe | unsafe; }
```

- **normal**(implicit): articolele sunt ambalate în poziția lor implicită ca și cum nu ar fi fost setată nicio valoare.
- **flex-start/ start:** articole ambalate la începutul containerului. (mai acceptat) flex-start respectă în flex-direction timp ce start respectă writing-mode direcția.
- **flex-end/ end:** articole ambalate până la capătul containerului. (mai mult suport) flex-end onorează în flex-direction timp ce capătul onorează writing-mode direcția.
- **center:** elemente centrate în container
- **space-between:** elemente distribuite uniform; prima linie este la începutul containerului, iar ultima este la sfârșit
- **space-around:** elemente distribuite uniform cu spațiu egal în jurul fiecărei linii
- **space-evenly** elementele sunt distribuite uniform, cu spațiu egal în jurul lor
- **stretch** liniile se întind pentru a ocupa spațiul rămas



Proprietatea **gap** controlează explicit spațiul dintre elementele flexibile. Aplică această spațiere doar între elementele care nu se află pe marginile exterioare. (gol, gol între rânduri, gol între coloane)

```
.container { display: flex; ... gap: 10px; gap: 10px 20px; /* row-gap column gap */  
row-gap: 10px; column-gap: 20px; }
```



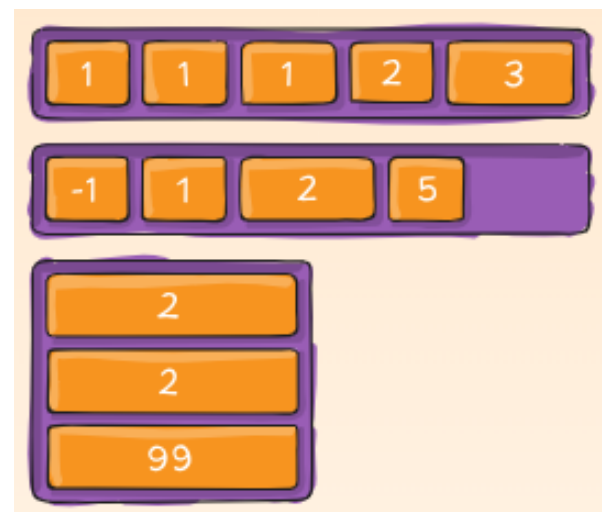
Proprietăți pentru copii (articole flexibile)

Order: În mod implicit, elementele flex sunt așezate în ordinea sursă.

Cu toate acestea, proprietatea **order** (comanda) controlează ordinea în care acestea apar în containerul flex.

```
.item { order: 5; /* default is 0 */ }
```

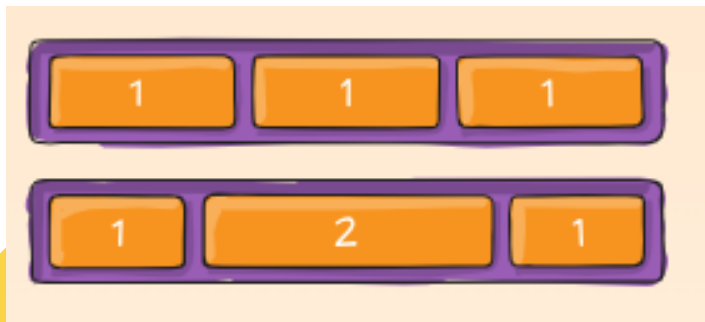
Elementele cu aceeași valoare order revin la ordinea sursă.



Flex-grow: Aceasta definește capacitatea unui element flexibil de a crește dacă este necesar. Acceptă o valoare fără unități care servește drept proporție. Diktează ce cantitate din spațiul disponibil în interiorul containerului flexibil ar trebui să ocupe elementul.

Dacă toate elementele au flex-grow valoarea 1, spațiul rămas în container va fi distribuit în mod egal tuturor copiilor. Dacă unul dintre copii are valoarea 2, acel copil va ocupa de două ori mai mult spațiu decât oricare dintre ceilalți (sau va încerca, cel puțin). Numerele negative sunt invalide.

```
.item { flex-grow: 4; /* default 0 */ }
```



Flex-shrink: Aceasta definește capacitatea unui element flexibil de a se micșora dacă este necesar. Numerele negative sunt invalide.

```
.item { flex-shrink: 3; /* default 1 */ }
```

Flex-basis: Aceasta definește dimensiunea implicită a unui element înainte ca spațiul rămas să fie distribuit. Poate fi o lungime (de exemplu, 20%, etc.) sau un cuvânt cheie. Cuvântul autocheie înseamnă „uită-te la proprietatea mea lățime sau înălțime” (ceea ce era făcut temporar de main-size cuvântul cheie până când a fost depreciat). Content Cuvântul cheie înseamnă „dimensionează-l în funcție de conținutul elementului” – acest cuvânt cheie nu este încă bine suportat, așa că este greu de testat și mai greu de știut ce fac frații săi max-content, min-content, și fit-content.

```
.item { flex-basis: | auto; /* default auto */ }
```

Dacă este setat la 0, spațiul suplimentar din jurul conținutului nu este luat în considerare. Dacă este setat la auto, spațiul suplimentar este distribuit în funcție de flex-grow valoarea sa.

Flex: Aceasta este prescurtarea pentru flex-grow, flex-shrink și flex-basis combinate. Al doilea și al treilea parametru (flex-shrink și flex-basis) sunt opționale. Implicit este 0 1 auto, dar dacă îl setați cu o singură valoare numerică, cum ar fi flex: 5;, aceasta schimbă flex-basis la 0%, deci este ca și cum ați seta flex-grow: 5; flex-shrink: 1; flex-basis: 0%;.

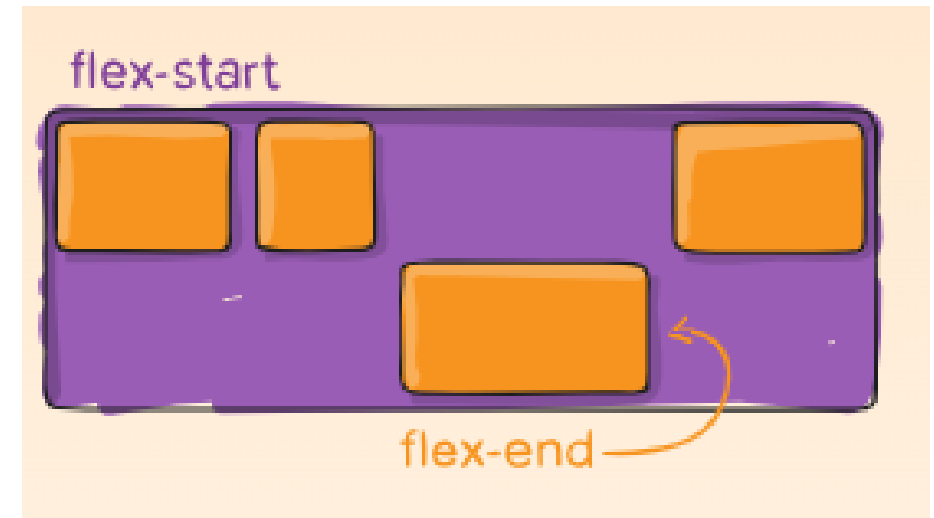
```
.item { flex: none | [ <'flex-grow'> <'flex-shrink'>? || <'flex-basis'> ] }
```

Se recomandă utilizarea acestei proprietăți prescurtate în loc să setați proprietățile individuale. Prescurtarea setează celelalte valori în mod inteligent.

Align-self: Aceasta permite suprascrierea alinierii implicite (sau a celei specificate) pentru elementele flexibile individuale.

```
.item { align-self: auto | flex-start | flex-end | center | baseline | stretch; }
```

Rețineți că float, clear și vertical-align nu au niciun efect asupra unui element flexibil.



CSS Grid

CSS Grid Layout (cunoscut și sub numele de „Grid” sau „CSS Grid”) este un sistem de aspect bidimensional bazat pe grilă care, în comparație cu orice sistem de aspect web din trecut, schimbă complet modul în care proiectăm interfețele utilizator. CSS a fost întotdeauna folosit pentru a aranja paginile noastre web, dar niciodată nu a făcut o treabă prea bună. Inițial, am folosit tabele, apoi elemente cu flotare, poziționare și blocuri inline, dar toate aceste metode au fost în esență trucuri și au omis o mulțime de funcționalități importante (de exemplu, centrarea verticală).

Proprietăți ale grilei CSS

display: Definește elementul ca un container de grilă și stabilește un nou context de formatare a grilei pentru conținutul său.

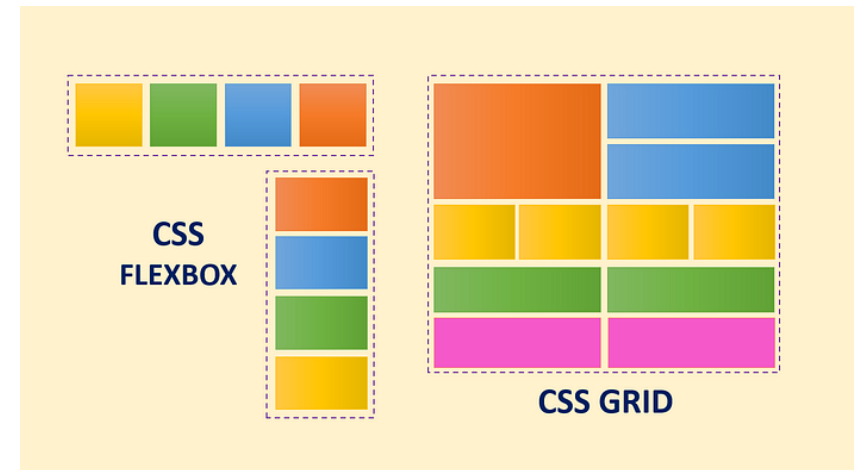
Valori:

- **grid** - Generează o grilă la nivel de bloc
- **inline-grid** - Generează o grilă la nivel de linie

```
.container { display: grid | inline-grid; }
```

Elementul asupra căruia **display: grid** se aplică. Este părintele direct al tuturor elementelor grilei. În acest exemplu container este containerul grilei.

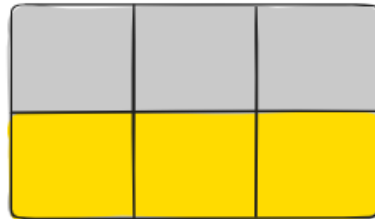
```
<div class="container"> <div class="item item-1"> </div> <div class="item  
item-2"> </div> <div class="item item-3"> </div> </div>
```



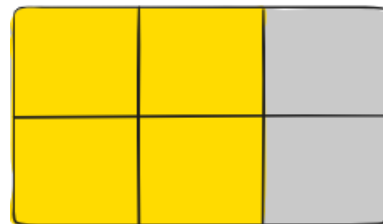
Linie de grid: Liniile de demarcație care alcătuiesc structura grid-ului. Acestea pot fi fie verticale („linii de grid pentru coloane”), fie orizontale („linii de grid pentru rânduri”) și se află de o parte și de alta a unui rând sau a unei coloane. Aici, linia galbenă este un exemplu de linie de grid pentru coloane.



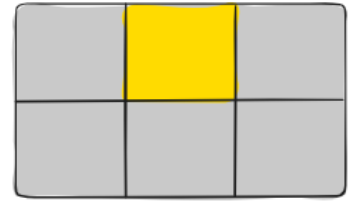
Pistă de grid: Spațiul dintre două linii de grid adiacente. Le puteți considera coloane sau rânduri ale grid-ului. Iată pista grid-ului dintre liniile grid-ului de pe al doilea și al treilea rând.



Zona grid-ului: Spațiul total înconjurat de patru linii de grid. O zonă de grid poate fi compusă din orice număr de celule de grid. Iată zona de grid dintre liniile de grilă 1 și 3 pentru rânduri și liniile de grid 1 și 3 pentru coloane.



Celulă de grid: Spațiul dintre două linii de grid de tip rând adiacent și două linii de grid de tip coloană adiacente. Este o singură „unitate” a grid-ului. Iată celula grid=ului dintre liniile de grid de tip rând 1 și 2 și liniile de grid de tip coloană 2 și 3.



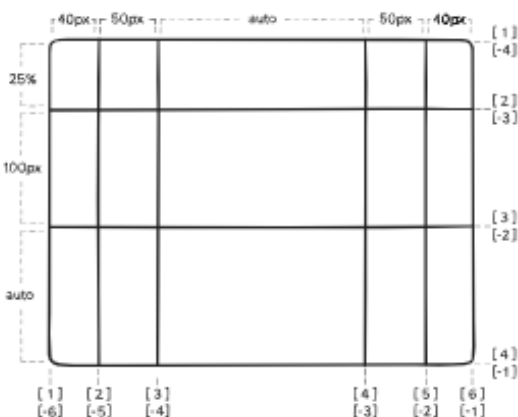
Grid-template-columns: Definește coloanele și rândurile grid-ului cu o listă de valori separate prin spații. Valorile reprezintă dimensiunea pistei, iar spațiul dintre ele reprezintă linia grid-ului.

Valori:

- **<track-size>** Poate fi o lungime, un procent sau o fracțiune din spațiul liber din grid folosind unitatea **fr**.
- **<line-name>** Un nume arbitrar la alegere.

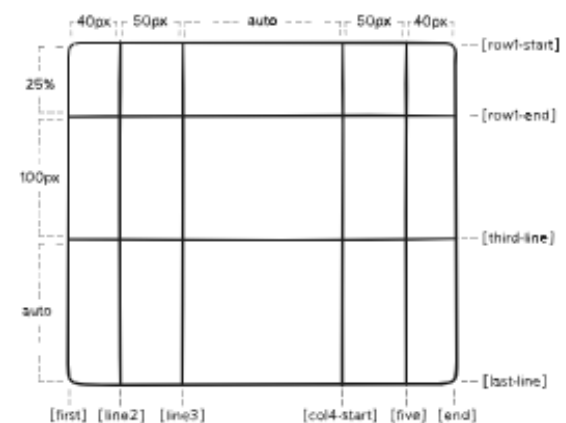
```
.container { grid-template-columns: ... ...; /* e.g. 1fr 1fr minmax(10px, 1fr) 3fr repeat(5, 1fr) 50px auto 100px 1fr */
grid-template-rows: ... ...; /* e.g. min-content 1fr min-content 100px 1fr max-content */ }
```

Liniile grid-ului primesc automat numere pozitive din aceste atribuiri (-1 fiind o alternativă pentru ultimul rând).



```
.container { grid-template-columns: [first] 40px [line2] 50px
[line3] auto [col4-start] 50px [five] 40px [end]; grid-template-
rows: [row1-start] 25% [row1-end] 100px [third-line] auto [last-
line]; }
```

```
.container { grid-template-rows: [row1-start] 25% [row1-end
row2-start] 25% [row2-end]; }
```



Unitatea **fr** vă permite să setați dimensiunea unei piste ca o fracțiune din spațiul liber al containerului grilei. De exemplu, aceasta va seta fiecare element la o treime din lățimea containerului grilei:

```
.container { grid-template-columns: 1fr 1fr 1fr; }
```

Spațiul liber este calculat după adăugarea oricăror elemente neflexibile. În acest exemplu, spațiul total liber disponibil unităților **fr** nu include cei 50px:

```
.container { grid-template-columns: 1fr 50px 1fr 1fr; }
```

O prescurtare pentru setarea `grid-template-rows`, `grid-template-columns`, și `grid-template-areas` într-o singură declarație. Valori:

- **None** - Setează toate cele trei proprietăți la valorile lor inițiale.
- **<grid-template-rows> / <grid-template-columns>**: Setează `grid-template-columns` și `grid-template-rows` la valorile specificate, respectiv, și setează `grid-template-areas` la none

```
.container { grid-template: none | <grid-template-rows> / <grid-template-columns>; }
```

De asemenea, acceptă o sintaxă mai complexă, dar destul de practică, pentru specificarea tuturor celor trei. Iată un exemplu:

```
.container { grid-template: [row1-start] "header header header" 25px [row1-end] [row2-start]  
"footer footer footer" 25px [row2-end] / auto 50px auto; }
```

Întrucât `grid-template` nu resetează proprietățile implicite ale grilei (`grid-auto-columns`, `grid-auto-rows`, și `grid-auto-flow`), ceea ce este probabil ceea ce doriți să faceți în majoritatea cazurilor, este recomandat să utilizați `grid` proprietatea în loc de `grid-template`.

grid-template-areas: Definește un șablon de grilă prin referire la numele zonelor de grilă specificate cu `grid-area` proprietatea. Repetarea numelui unei zone de grilă face ca respectivul conținut să se întindă pe acele celule. Punct semnifică o celulă goală. Sintaxa în sine oferă o vizualizare a structurii grilei.

Valori:

- **<grid-area-name>** Numele unei zone de grilă specificate `grid-area` cu .
- **.** - Un punct semnifică o celulă goală în grilă. `.container { grid-template-areas: "<grid-area-name> | . | none | ..." "..."; }`
- **None** - Nu sunt definite zone de grid.

Exemplu:

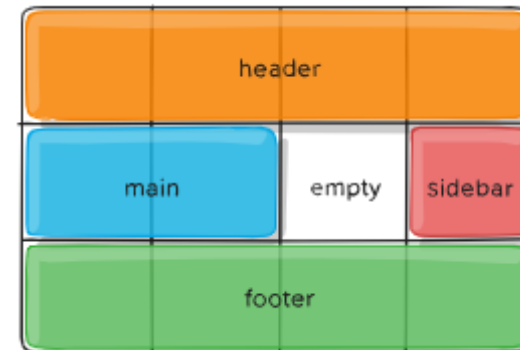
```
.item-a { grid-area: header; } .item-b { grid-area: main; } .item-c { grid-area: sidebar; } .item-d { grid-area: footer; } .container { display: grid; grid-template-columns: 50px 50px 50px 50px; grid-template-rows: auto; grid-template-areas: "header header header header" "main main . sidebar" "footer footer footer footer"; }
```

Aceasta va crea un grid cu patru coloane lățiime și trei rânduri înălțime. Întregul rând de sus va fi compus din zona **antetului** . Rândul din mijloc va fi compus din două zone **principale** , o celulă goală și o zonă **de bară laterală** . Ultimul rând este în întregime **subsol** .

Fiecare rând din declarația ta trebuie să aibă același număr de celule.

Puteți folosi oricâte puncte adiacente pentru a declara o singură celulă goală. Atâta timp cât punctele nu au spații între ele, acestea reprezintă o singură celulă.

Observați **că** nu denumiți linii cu această sintaxă, ci doar zone. Când utilizați această sintaxă, liniile de la ambele **capete** ale zonelor sunt de fapt denumite automat. Dacă numele zonei grilei este **foo** , numele rândului de început și al coloanei de început a zonei va fi **foo -start** , iar numele ultimei linii de rând și al ultimei linii de coloană va fi **foo -end** . Aceasta înseamnă că unele linii pot avea mai multe nume, cum ar fi linia din extrema stângă din exemplul de mai sus, care va avea trei nume: header-start, main-start, și footer-start.



Grid: O prescurtare pentru setarea tuturor următoarelor proprietăți într-o singură declarație: `grid-template-rows`, `grid-template-columns`, `grid-template-areas`, `grid-auto-rows`, `grid-auto-columns`, și `grid-auto-flow`. Puteți specifica doar proprietățile explicite sau implicite ale grilei într-o singură declarație de grilă.

Valori:

- **None** - Setează toate subproprietățile la valorile lor inițiale.
- **<grid-template>** - Funcționează la fel ca `grid-template` prescurtarea.
- **<grid-template-rows> / [auto-flow && dense?] <grid-auto-columns>?:** Setează `grid-template-rows` la valoarea specificată. Dacă `auto-flow` cuvântul cheie se află în dreapta barei oblice, acesta setează `grid-auto-flow` la `column`. Dacă `dense` cuvântul cheie este specificat suplimentar, algoritmul de plasare automată utilizează un algoritm de împachetare „densă”. Dacă `grid-auto-columns` este omis, este setat la `auto`.
- **[auto-flow && dense?] <grid-auto-rows>? / <grid-template-columns>:** Setează `grid-template-columns` la valoarea specificată. Dacă `auto-flow` cuvântul cheie se află la stânga barei oblice, acesta setează `grid-auto-flow` la `row`. Dacă `dense` cuvântul cheie este specificat suplimentar, algoritmul de plasare automată utilizează un algoritm de împachetare „densă”. Dacă `grid-auto-rows` este omis, este setat la `auto`.

Exemple:

Următoarele două blocuri de cod sunt echivalente:

```
.container { grid: 100px 300px / 3fr 1fr; }  
.container { grid-template-rows: 100px 300px; grid-template-columns: 3fr 1fr; }
```

```
.container { grid: auto-flow / 200px 1fr; }
```

```
.container { grid-auto-flow: row; grid-template-columns: 200px 1fr; }
```

```
.container { grid: auto-flow dense 100px / 1fr 2fr; }
```

```
.container { grid-auto-flow: row dense; grid-auto-rows: 100px; grid-template-columns: 1fr 2fr; }
```

```
.container { grid: 100px 300px / auto-flow 200px; }
```

```
.container { grid-template-rows: 100px 300px; grid-auto-flow: column; grid-auto-columns: 200px; }
```

De asemenea, acceptă o sintaxă mai complexă, dar destul de practică, pentru setarea tuturor elementelor simultan. Specificați `grid-template-areas`, `grid-template-rows` și `grid-template-columns`, iar toate celelalte subproprietăți sunt setate la valorile lor inițiale. Ceea ce faceți este să specificați numele liniilor și dimensiunile pistelor în linie cu zonele respective ale grilei. Cel mai ușor de descris este un exemplu:

```
.container { grid: [row1-start] "header header header" 1fr [row1-end] [row2-start] "footer footer footer" 25px [row2-end] / auto 50px auto; }
```

```
.container { grid-template-areas: "header header header" "footer footer footer"; grid-template-rows: [row1-start] 1fr [row1-end row2-start] 25px [row2-end]; grid-template-columns: auto 50px auto; }
```

Justify-items: Aliniază elementele grilei de-a lungul axei în linie (rând) (spre deosebire de `align-items` alinierea de-a lungul axei blocului (coloanei)). Această valoare se aplică tuturor elementelor grilei din interiorul containerului.

Valori:

- **stretch(implicit):** Umple întreaga lățime a celulei.
- **Start:** Aliniază elementele astfel încât să fie la același nivel cu marginea inițială a celulei lor
- **End:** Aliniază elementele astfel încât să fie la același nivel cu marginea de capăt a celulei lor
- **Center:** Aliniază elementele în centrul celulei lor

```
.container { justify-items: start | end | center | stretch; }
```

Exemplu:

```
.container { justify-items: start; }
```



```
.container { justify-items: end; }
```



```
.container { justify-items: center; }
```



```
.container { justify-items: stretch; }
```



Acest comportament poate fi setat și pentru elemente individuale ale grilei prin intermediul `justify-self` proprietății.

Align-items: Aliniază elementele grilei de-a lungul axei blocului (coloanei) (spre deosebire justify-items de alinierea de-a lungul axei în linie (rândului)). Această valoare se aplică tuturor elementelor grilei din interiorul containerului.

Valori:

- **stretch(implicit):** Umple întreaga înălțime a celulei (aceasta este valoarea implicită)
- **Start** - Aliniază elementele astfel încât să fie la același nivel cu marginea inițială a celulei lor
- **End** - Aliniază elementele astfel încât să fie la același nivel cu marginea de capăt a celulei lor
- **Center** - Aliniază elementele în centrul celulei lor
- **baseline:** Aliniază elementele de-a lungul liniei de bază a textului . Există modificatori pentru baseline— first baseline și last baseline care vor utiliza linia de bază de la prima sau ultima linie în cazul textului cu mai multe linii.

```
.container { align-items: start | end | center | stretch; }
```

Exemple:

```
.container { align-items: start; }
```



```
.container { align-items: end; }
```



```
.container { align-items: center; }
```



```
.container { align-items: stretch; }
```



Place-items: place-items setează ambele proprietăți align-items și justify-items într-o singură declarație.

Valori:

- <align-items> / <justify-items>: Prima valoare setează align-items, a doua valoare justify-items. Dacă a doua valoare este omisă, prima valoare este atribuită ambelor proprietăți.

Acest lucru poate fi foarte util pentru o centrare multidirecțională super rapidă:

```
.center { display: grid; place-items: center; }
```

Justify-content: Uneori, dimensiunea totală a grilei poate fi mai mică decât dimensiunea containerului său de grilă. Acest lucru se poate întâmpla dacă toate elementele grilei sunt dimensionate cu unități neflexibile, cum ar fi px. În acest caz, puteți seta alinierea grilei în cadrul containerului grilei. Această proprietate aliniază grila de-a lungul axei în linie (rând) (spre deosebire de align-content aceasta, care aliniază grila de-a lungul axei blocului (coloanei)).

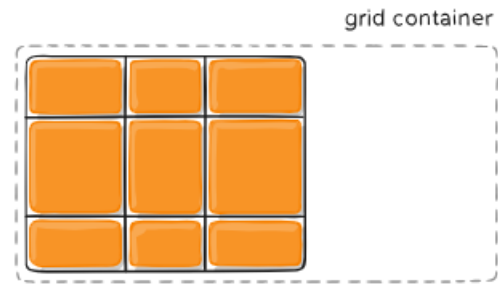
Valori:

- **Start** - Aliniază grila astfel încât să fie la același nivel cu marginea inițială a containerului grilei.
- **End** - Aliniază grila astfel încât să fie la același nivel cu marginea de capăt a containerului grilei.
- **Center** - Aliniază grila în centrul containerului grilei.
- **Stretch** - Redimensionează elementele grilei pentru a permite grilei să umple întreaga lățime a containerului grilei.
- **space-around** - Plasează un spațiu egal între fiecare element al grilei, cu spații egale cu jumătate de dimensiune la capetele îndepărtate.
- **space-between** - Plasează un spațiu egal între fiecare element al grilei, fără spațiu la capetele îndepărtate.
- **space-evenly** - Plasează o cantitate uniformă de spațiu între fiecare element al grilei, inclusiv capetele îndepărtate.

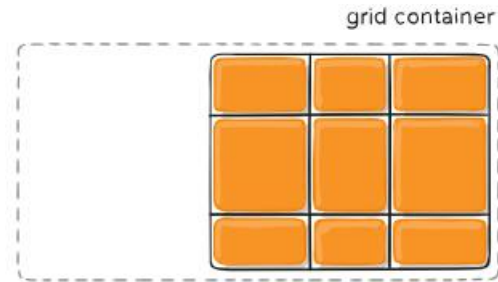
```
.container { justify-content: start | end | center | stretch | space-around | space-between | space-evenly; }
```

Exemple:

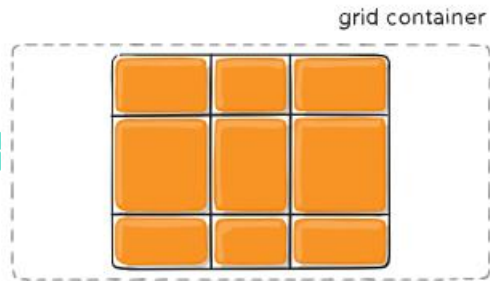
`.container { justify-content: start; }`



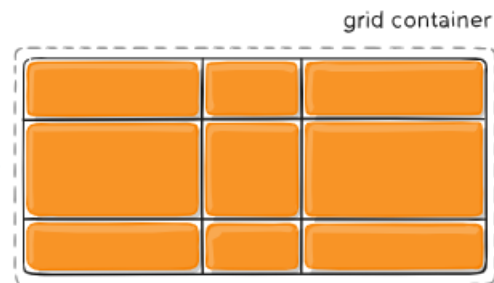
`.container { justify-content: end; }`



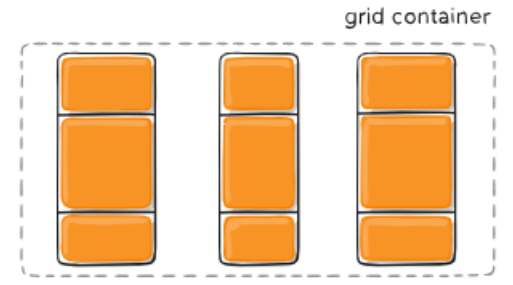
`.container { justify-content: center; }`



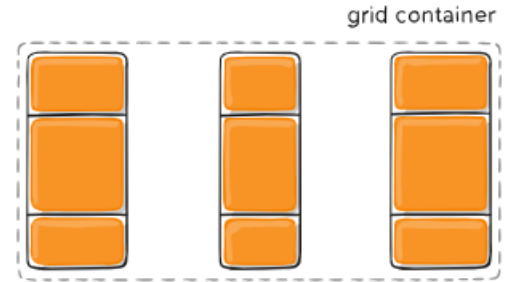
`.container { justify-content: stretch; }`



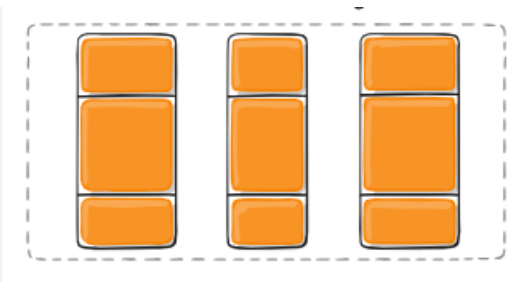
`.container { justify-content: space-around; }`



`.container { justify-content: space-between; }`



`.container { justify-content: space-evenly; }`



Align-content: Uneori, dimensiunea totală a grilei poate fi mai mică decât dimensiunea containerului său de grilă. Acest lucru se poate întâmpla dacă toate elementele grilei sunt dimensionate cu unități neflexibile, cum ar fi px. În acest caz, puteți seta alinierea grilei în cadrul containerului grilei. Această proprietate aliniază grila de-a lungul axei blocului (coloanei) (spre deosebire de justify-content aceasta, care aliniază grila de-a lungul axei în linie (rândului)).

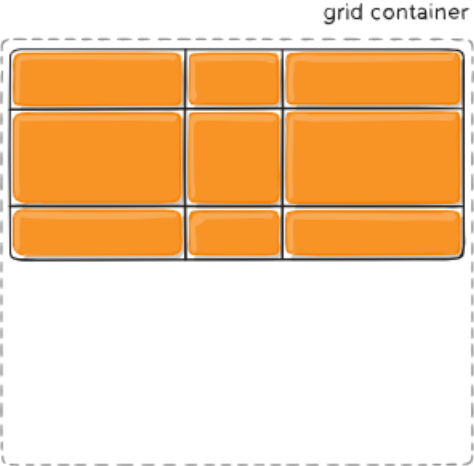
Valori:

- **Start** Aliniază grila astfel încât să fie la același nivel cu marginea inițială a containerului grilei.
- **End** Aliniază grila astfel încât să fie la același nivel cu marginea de capăt a containerului grilei.
- **Center** Aliniază grila în centrul containerului grilei.
- **Stretch** Redimensionează elementele grilei pentru a permite grilei să umple întreaga înălțime a containerului grilei.
- **space-around** Plasează un spațiu egal între fiecare element al grilei, cu spații egale cu jumătate de dimensiune la capetele îndepărtate.
- **space-between** Plasează un spațiu egal între fiecare element al grilei, fără spațiu la capetele îndepărtate.
- **space-evenly** Plasează o cantitate uniformă de spațiu între fiecare element al grilei, inclusiv capetele îndepărtate.

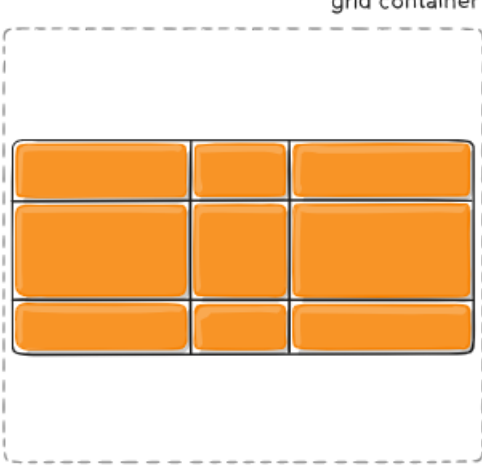
```
.container { align-content: start | end | center | stretch |  
space-around | space-between | space-evenly; }
```

Exemple:

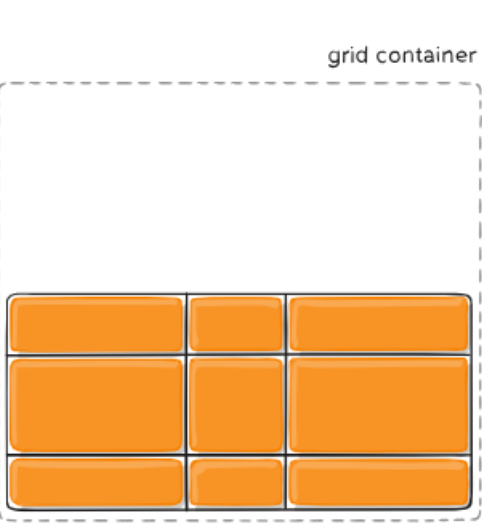
```
.container { align-content: start; }
```



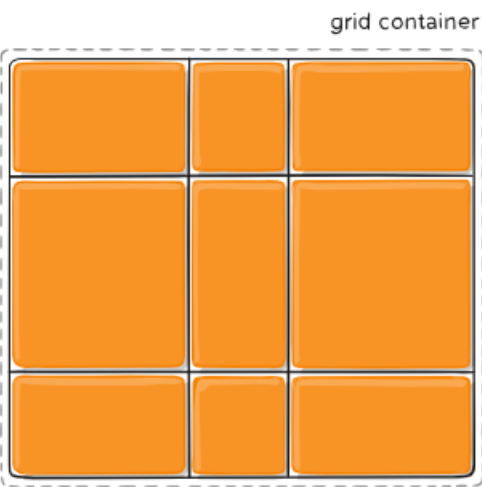
```
.container { align-content: center; }
```



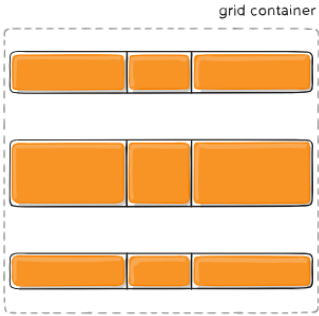
```
.container { align-content: end; }
```



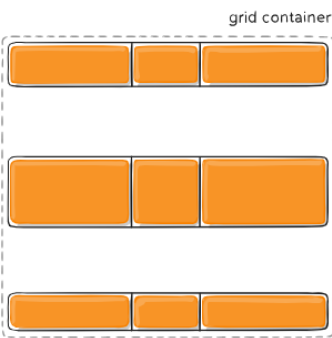
```
.container { align-content: stretch; }
```



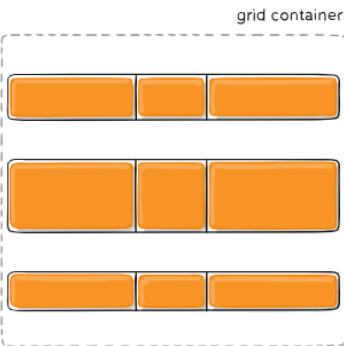
```
.container { align-content: space-around; }
```



```
.container { align-content: space-between; }
```



```
.container { align-content: space-evenly; }
```



Grid-auto-*: Specifică dimensiunea oricăror piste de grilă generate automat (cunoscute și ca **piste de grilă implicite**). Pistele implicite sunt create atunci când există mai multe elemente de grilă decât celule în grilă sau când un element de grilă este plasat în afara grilei explicite.

Valori: **<track-size>** Poate fi o lungime, un procent sau o fracțiune din spațiul liber din grilă (folosind unitatea **fr**).

```
.container { grid-auto-columns: <track-size> ...; grid-auto-rows: <track-size> ...; }
```

Pentru a ilustra cum se creează grilele implicite:

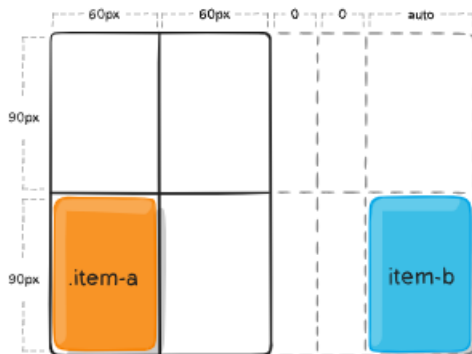
```
.container { grid-template-columns: 60px 60px; grid-template-rows: 90px 90px; }
```

Aceasta creează o grilă de 2×2.

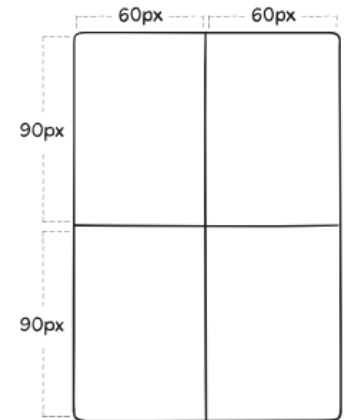
Dar acum imaginează-ți că folosești **grid-column** și **grid-row** pentru a poziționa elementele grilei astfel:

```
.item-a { grid-column: 1 / 2; grid-row: 2 / 3; }
```

```
.item-b { grid-column: 5 / 6; grid-row: 2 / 3; }
```

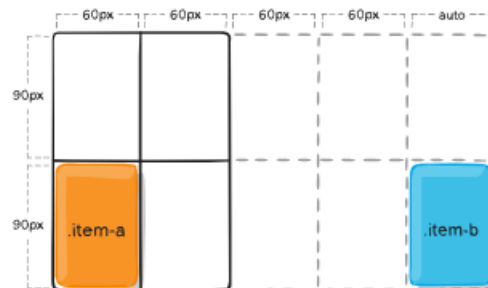


Am spus **.item-b** să începem la linia 5 a coloanei și să terminăm la linia 6 a coloanei, *dar nu am definit niciodată o linie 5 sau 6*. Deoarece am făcut referire la linii care nu există, sunt create piste implicite cu lățimi de 0 pentru a umple golurile.



Putem folosi `grid-auto-columns` și `grid-auto-rows` pentru a specifica lățimile acestor piste implicite:

```
.container { grid-auto-columns: 60px; }
```



Grid-auto-flow: Dacă aveți elemente din grilă pe care nu le plasați explicit pe grilă, algoritmul de plasare automată se activează pentru a plasa automat elementele. Această proprietate controlează modul în care funcționează algoritmul de plasare automată.

Valori:

- **Row** Indică algoritmului de plasare automată să completeze fiecare rând pe rând, adăugând rânduri noi după cum este necesar (implicit)
- **column:** Indică algoritmului de plasare automată să completeze fiecare coloană pe rând, adăugând coloane noi după cum este necesar
- **Dense** Indică algoritmului de plasare automată să încerce să umple golurile mai devreme în grilă dacă elemente mai mici apar mai târziu

```
.container { grid-auto-flow: row | column | row dense | column dense; }
```

Rețineți că **dense** modifică doar ordinea vizuală a elementelor și ar putea face ca acestea să apară în afara ordinii, ceea ce este dăunător pentru accesibilitate.

Exemple:

```
<section class="container">
<div class="item-a">item-a </div>
<div class="item-b">item-b</div>
<div class="item-c">item-c</div>
<div class="item-d">item-d</div>
<div class="item-e">item-e</div>
</section>
```

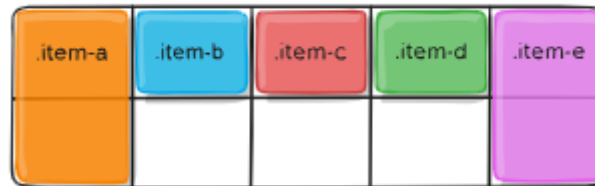
Definiți o grilă cu cinci coloane și două rânduri și setați `grid-auto-flow` la `row` (care este și valoarea implicită):

```
.container { display: grid; grid-template-columns: 60px 60px 60px 60px 60px;
grid-template-rows: 30px 30px; grid-auto-flow: row; }
```

Când plasați elementele pe grilă, specificați locuri doar pentru două dintre ele:

```
.item-a { grid-column: 1; grid-row: 1 / 3; } .item-e { grid-column: 5; grid-row: 1 / 3; }
```

Deoarece am setat `grid-auto-flow` la `row`, grila noastră va arăta astfel. Observați cum cele trei elemente pe care nu le-am plasat (`element-b` , `element-c` și `element-d`) se desfășoară pe rândurile disponibile:



Dacă în schimb setăm `grid-auto-flow`, `column item -b`, `item-c` și `item-d` se desfășoară în coloane:

```
.container { display: grid; grid-template-columns: 60px 60px 60px 60px 60px;  
60px; grid-template-rows: 30px 30px; grid-auto-flow: column; }
```

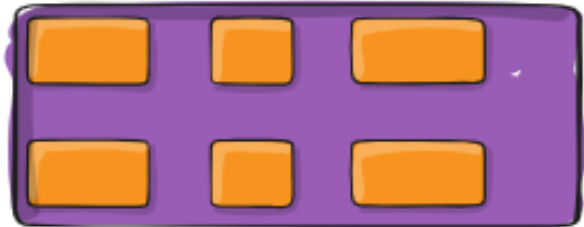


Proprietatea **gap** controlează în mod explicit spațiul dintre elementele flexibile. Aplică acea spațiere doar între elementele care nu se află pe marginile exterioare. Este o prescurtare care combină proprietățile `row-gap` și `column-gap`.

gap: 10px



gap: 30px



gap: 10px 30px



```
.container { display: flex;  
/* ... */  
gap: 10px; gap: 10px 20px;  
/* row-gap column gap */  
row-gap: 10px; column-gap: 20px; }
```

Funcții de dimensionare

- **fit-content()** Folosește spațiul disponibil, dar niciodată mai puțin decât min-content și niciodată mai mult decât max-content.
- **max()** Funcția CSS `max()` preia două sau mai multe argumente separate prin virgulă ale valorilor de spațiere, le evaluează și returnează cea mai mare valoare din grup .
- **min()** Funcția CSS `min()` preia două sau mai multe argumente separate prin virgulă ale valorilor de spațiere, le evaluează și returnează cea mai mică valoare din grup .
- **minmax()** Funcția CSS este utilizată exclusiv cu CSS Grid pentru a defini un interval de dimensiuni pentru piste grilei (adică rândurile și coloanele grilei). Vă permite să specificați două argumente - o dimensiune minimă și o dimensiune maximă pentru o pistă - permițând pistei grilei să crească și să se micșoreze în acel interval, în funcție de spațiul disponibil.
- **repeat()** Funcția `repeat()` poate economisi o parte din timp tastând:

```
grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr 1fr 1fr;  
/* easier: */ grid-template-columns: repeat(8, 1fr);  
/* especially when: */ grid-template-columns:  
repeat(8, minmax(10px, 1fr));
```

Dar **repeat()** poate deveni și mai sofisticat atunci când este combinat cu cuvinte cheie:

• **auto-fill** Încadrați cât mai multe coloane posibile pe un rând, chiar dacă acestea sunt goale.

• **auto-fit**: Încadrați în spațiu orice coloane există. Preferați extinderea coloanelor pentru a umple spațiul, în loc de coloane goale.

```
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
```

Subgrid este o caracteristică extrem de utilă a grilelor care permite elementelor grilei să aibă propria grilă ce moștenește liniile de grilă de la grila părinte.

```
.parent-grid { display: grid; grid-template-columns: repeat(9, 1fr); }  
.grid-item { grid-column: 2 / 7; display: grid; grid-template-columns: subgrid; }  
.child-of-grid-item { /* gets to participate on parent grid! */ grid-column: 3 / 6; }
```

De asemenea, este util să știi despre **display: contents;**. Aceasta nu este același lucru cu subgrid, dar poate fi un instrument util uneori într-un mod similar.

```
<div class="grid-parent">  
  <div class="grid-item"></div>  
  <div class="grid-item"></div>  
  <ul style="display: contents;">  
    <!-- These grid-items get to participate on the same grid!-->  
    <li class="grid-item"></li>  
    <li class="grid-item"></li>  
  </ul> </div>
```

Bootstrap Grid

Sistemul de grid **Bootstrap** oferă o modalitate ușoară și puternică de a crea layout-uri responsive de toate formele și dimensiunile. Este construit cu flexbox cu abordare mobile-first. De asemenea, este complet responsive și folosește un sistem cu doisprezece coloane (12 coloane disponibile pe rând) și șase niveluri responsive implicite.

Puteți utiliza clasele de grid predefinite din Bootstrap pentru a crea rapid machete pentru diferite tipuri de dispozitive, cum ar fi telefoane mobile, tablete, laptopuri, desktop-uri și așa mai departe.

De exemplu, puteți utiliza clasele **.col-*** pentru a crea coloane de grilă pentru dispozitive foarte mici, cum ar fi telefoanele mobile în modul portret și **.col-sm-*** clase pentru telefoane mobile în modul peisaj.

În mod similar, puteți utiliza **.col-md-*** clasele pentru a crea coloane de grilă pentru dispozitive cu ecran mediu, cum ar fi tabletele, clase **.col-lg-*** pentru dispozitive precum laptopuri mici, **.col-xl-*** clase pentru laptopuri și desktopuri și **.col-xxl-*** clase pentru ecrane mari de desktopuri.



Caracteristicile cheie ale sistemului Bootstrap

Features	X-Small (xs)	Small (sm)	Medium (md)	Large (lg)	X-Large (xl)	XX-Large (xxl)
Bootstrap Grid System	<576px	≥576px	≥768px	≥992px	≥1200px	≥1400px
Container max-width	None (auto)	540px	720px	960px	1140px	1320px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-	.col-xxl-
Number of columns	12					
Gutter width	1.5rem (.75rem on left and right)					
Custom gutters	Yes					
Nestable	Yes					
Column ordering	Yes					

Aplicarea oricărei **.col-sm-*** clase unui element va avea un efect nu numai asupra dispozitivelor mici, ci și asupra dispozitivelor medii, mari și foarte mari (lățime a viewport-ului $\geq 768\text{px}$), dacă nu este prezentă o clasă **.col-md-***, **.col-lg-***, **.col-xl-***, sau **.col-xxl-***.

În mod similar, **.col-md-*** clasa va avea efect nu doar asupra dispozitivelor medii, ci și asupra dispozitivelor mari și extra-mari, dacă nu este prezentă o clasă **.col-lg-***, **.col-xl-***, sau **.col-xxl-***

Acum se pune întrebarea cum se creează rânduri și coloane folosind acest sistem de grilă responsiv cu 12 coloane. Răspunsul este destul de simplu, mai întâi creați un container care acționează ca un wrapper pentru rânduri și coloane folosind orice clase de containere, cum ar fi **.container**, după aceea creați rânduri în interiorul containerului folosind **.rowclasa**, iar pentru a crea coloane în interiorul oricărui rând puteți utiliza clasele **.col-***, **.col-sm-***, **.col-md-***, **.col-lg-***, **.col-xl-*** și **.col-xxl-***.

Coloanele sunt zone de conținut propriu-zise unde vom plasa conținutul nostru. În secțiunile următoare vom pune toate aceste lucruri în practică și vom vedea cum funcționează de fapt:

Crearea machetelor cu două coloane

Următorul exemplu vă va arăta cum să creați machete cu două coloane pentru dispozitive medii, mari și foarte mari, cum ar fi tablete, laptopuri și desktopuri etc. Cu toate acestea, pe telefoanele mobile (lățimea ecranului mai mică de 768px), coloanele vor deveni automat orizontale (2 rânduri, 1 coloană).

```
<div class="container">
  <!--Row with two equal columns-->
  <div class="row"> <div class="col-md-6">Column
left</div>
<div class="col-md-6">Column right</div>
</div>
  <!--Row with two columns divided in 1:2 ratio-->
  <div class="row"> <div class="col-md-4">Column
left</div>
<div class="col-md-8">Column right</div> </div>
  <!--Row with two columns divided in 1:3 ratio-->
  <div class="row"> <div class="col-md-3">Column
left</div>
<div class="col-md-9">Column right</div>
</div> </div>
```

Notă: Într-un layout de tip grilă, conținutul trebuie plasat în interiorul coloanelor (`.col` și `.col-*`) și numai coloanele pot fi copii direcți ai rândurilor (`.row`). De asemenea, rândurile ar trebui plasate într-un container (fix sau fluid) pentru o umplură și o aliniere corecte.

Lățimile coloanelor grilei sunt setate în procente, deci sunt întotdeauna fluide și dimensionate în raport cu elementul părinte. În plus, fiecare coloană are o zonă de umplere orizontală (numită *spatiu*) pentru controlul spațiului dintre coloanele individuale.

Întrucât sistemul de Bootstrap se bazează pe 12 coloane, pentru a păstra coloanele pe o singură linie (adică una lângă alta), suma numerelor coloanelor grilei dintr-un singur rând nu ar trebui să fie mai mare de 12.

Crearea unor machete cu trei coloane

În mod similar, puteți crea alte machete bazate pe principiul de mai sus. De exemplu, următorul exemplu va crea de obicei machete cu trei coloane pentru ecranele laptopurilor și desktopurilor. Funcționează și pe tablete în modul peisaj dacă rezoluția ecranului este mai mare sau egală cu 992 pixeli (de exemplu, Apple iPad). Cu toate acestea, în modul portret, coloanele grilei vor fi orizontale ca de obicei.

```
<div class="container"> <!--Row with three equal columns-->
<div class="row"> <div class="col-lg-4">Column left</div> <div
class="col-lg-4">Column middle</div>
<div class="col-lg-4">Column right</div> </div> <!--Row with
three columns divided in 1:4:1 ratio-->
<div class="row"> <div class="col-lg-2">Column left</div>
<div class="col-lg-8">Column middle</div>
<div class="col-lg-2">Column right</div>
</div> <!--Row with three columns divided unevenly-->
  <div class="row"> <div class="col-lg-3">Column left</div>
<div class="col-lg-7">Column middle</div>
<div class="col-lg-2">Column right</div> </div> </div>
```

Coloane cu aspect automat Bootstrap

De asemenea, puteți crea coloane cu lățime egală pentru toate dispozitivele (x-small, small, medium, large, x-large și xx-large) utilizând pur și simplu clasa .col, fără a specifica niciun număr de coloană.

Să încercăm următorul exemplu pentru a înțelege cum funcționează exact:

```
<div class="container"> <!--Row with two equal columns-->
<div class="row"> <div class="col">Column one</div>
<div class="col">Column two</div>
</div> <!--Row with three equal columns-->
<div class="row"> <div class="col">Column one</div>
  <div class="col">Column two</div>
  <div class="col">Column three</div> </div> </div>
```

În plus, puteți seta lățimea unei coloane și puteți permite coloanelor surori să se redimensioneze automat în mod egal în jurul acesteia. Puteți utiliza clasele de grilă predefinite sau lățimile în linie.

Comportamentul de încadrare a coloanelor

Acum vom crea machete mai flexibile care modifică orientarea coloanelor în funcție de dimensiunea ferestrei de vizualizare. Următorul exemplu va crea o machetă cu trei coloane pe dispozitive mari, cum ar fi laptopuri și desktopuri, precum și pe tablete (de exemplu, Apple iPad) în modul peisaj, dar pe dispozitive medii, cum ar fi tabletele, în modul portret ($768\text{px} \leq \text{lățimea ecranului} < 992\text{px}$), se va transforma într-o machetă cu două coloane, în care a treia coloană se mută în partea de jos a primelor două coloane.

```
<div class="container"> <div class="row"> <div class="col-md-4  
col-lg-3">Column one</div>  
<div class="col-md-8 col-lg-6">Column two</div>  
<div class="col-md-12 col-lg-3">Column three</div> </div> </div>
```

După cum puteți vedea în exemplul de mai sus, suma numerelor coloanelor grilei medii (adică col-md-*) este $3 + 9 + 12 = 24 > 12$, prin urmare, al treilea <div>element din clasa .col-md-12 care adaugă coloanele suplimentare dincolo de maximum 12 coloane într-o .row, este înfășurat pe o linie nouă ca o unitate contiguă pe dispozitivele cu ecran mediu.

Crearea de machete cu mai multe coloane cu Bootstrap

Cu noul sistem de grid flexbox Bootstrap mobile first, puteți controla cu ușurință modul în care aspectul site-ului dvs. web va fi afișat pe diferite tipuri de dispozitive cu dimensiuni diferite ale ecranului sau ferestrei de vizualizare, cum ar fi telefoane mobile, tablete, desktop-uri etc. Să luăm în considerare următoarea ilustrație.



În ilustrația de mai sus există un total de 12 casete de conținut pe toate dispozitivele, dar amplasarea acestora variază în funcție de dimensiunea ecranului dispozitivului. La fel ca pe dispozitivele mobile, aspectul este redat ca o grilă pe o coloană, cu o coloană și 12 rânduri plasate una deasupra celeilalte, în timp ce pe tablete este redat ca o grilă pe două coloane, cu 2 coloane și 6 rânduri.

În plus, pe dispozitivele cu ecran mare, cum ar fi laptopurile și desktopurile, acesta este redat ca o grilă cu trei coloane, care are 3 coloane și 4 rânduri, iar în final, pe dispozitivele cu ecran foarte mare, cum ar fi desktopurile mari, acesta este redat ca o grilă cu patru coloane, care are 4 coloane și 3 rânduri.

Acum întrebarea este cum putem crea astfel de layout-uri responsive folosind acest sistem de grilă Bootstrap flexbox. Să începem cu dispozitivul țintă principal. Să presupunem că dispozitivul nostru țintă principal este un laptop sau un desktop normal. Deoarece layout-ul laptopului nostru are 3 coloane și 4 rânduri, adică un layout de grilă de 3x4, codul HTML pentru crearea unei astfel de structuri de grilă ar arăta cam așa.

```
<div class="container-lg"> <div class="row">
<div class="col-xl-4"><p>Box 1</p></div> <div
class="col-xl-4"><p>Box 2</p></div> <div
class="col-xl-4"><p>Box 3</p></div> <div
class="col-xl-4"><p>Box 4</p></div> <div
class="col-xl-4"><p>Box 5</p></div> <div
class="col-xl-4"><p>Box 6</p></div> <div
class="col-xl-4"><p>Box 7</p></div> <div
class="col-xl-4"><p>Box 8</p></div> <div
class="col-xl-4"><p>Box 9</p></div> <div
class="col-xl-4"><p>Box 10</p></div> <div
class="col-xl-4"><p>Box 11</p></div> <div
class="col-xl-4"><p>Box 12</p></div> </div>
</div>
```

Clasa **.container-lg** face containerul 100% lățime dacă lățimea viewport-ului este mai mică de 992px, utilizând astfel întreaga lățime disponibilă pe ecrane mai mici.

Dacă vedeți rezultatul exemplului de mai sus pe un dispozitiv mare, cum ar fi un laptop sau un desktop, care are o lățime a ecranului sau a viewport-ului mai mare sau egală cu 1200px, dar mai mică de 1400px, veți constata că aspectul are 4 rânduri, unde fiecare rând are 3 coloane egale, rezultând un aspect de grilă de 3x4.

Acum este momentul să personalizăm layout-ul nostru pentru alte dispozitive. Să începem prin a-l personaliza pentru dispozitive medii, cum ar fi tabletele ($768\text{px} \leq \text{lățime viewport} < 1200\text{px}$). Deoarece pe tabletă layout-ul nostru este redat ca grile 2x6 (adică 2 coloane și 6 rânduri), putem adăuga clasa `.col-md-6` pe fiecare coloană.

```
<div class="container-lg"> <div class="row">
  <div class="col-xl-4 col-md-6"><p>Box 1</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 2</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 3</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 4</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 5</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 6</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 7</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 8</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 9</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 10</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 11</p></div>
  <div class="col-xl-4 col-md-6"><p>Box 12</p></div> </div> </div>
```

Pentru comoditate, alegeți dispozitivul țintă principal și creați mai întâi un layout pentru acel dispozitiv, după care adăugați clase pentru a-l face responsiv pentru alte dispozitive.

Imbricarea coloanelor grid-ului

Coloanele grid Bootstrap sunt, de asemenea, imbricabile, ceea ce înseamnă că puteți plasa rânduri și coloane în interiorul unei coloane existente. Cu toate acestea, formula pentru plasarea coloanelor va fi aceeași, adică suma numerelor coloanelor trebuie să fie egală cu 12 sau mai mică într-un singur rând.

```
<div class="container"> <div class="row"> <div class="col-sm-8">Column left</div>
<div class="col-sm-4"> <!--Column right with nested rows and columns-->
<div class="row"> <div class="col-12"></div> </div>
<div class="row"> <div class="col-6"></div>
<div class="col-6"></div> </div> </div> </div> </div>
```

Crearea coloanelor cu lățime variabilă

Puteți utiliza **col-{breakpoint}-auto** clasele pentru a dimensiona coloanele în funcție de lățimea naturală a conținutului lor. Încercați următorul exemplu pentru a vedea cum funcționează:

```
<div class="container"> <div class="row justify-content-md-center">
<div class="col-md-3">Column left</div>
<div class="col-md-auto">Variable width column</div>
  <div class="col-md-3">Column right</div> </div>
<div class="row"> <div class="col">Column left</div>
  <div class="col-auto">Variable width column</div>
  <div class="col">Column right</div> </div> </div>
```

Alinierea coloanelor grilei

Puteți utiliza utilitățile de aliniere flexbox pentru a alinia vertical și orizontal coloanele grilei în interiorul unui container. Încercați următoarele exemple pentru a înțelege cum funcționează:

Alinierea verticală a coloanelor grilei

Puteți utiliza clasele `.align-items-start`, `.align-items-center`, și `.align-items-end` pentru a alinia coloanele grilei vertical în partea de sus, mijloc și respectiv jos a unui container.

```
<div class="container"> <div class="row align-items-start"> <div  
class="col">Column one</div>  
<div class="col">Column two</div>  
<div class="col">Column three</div> </div>  
<div class="row align-items-center"> <div class="col">Column  
one</div> <div class="col">Column two</div>  
<div class="col">Column three</div> </div>  
<div class="row align-items-end"> <div class="col">Column one</div>  
<div class="col">Column two</div>  
<div class="col">Column three</div> </div> </div>
```

Coloanele individuale din interiorul unui rând pot fi aliniate și pe verticală. Iată un exemplu:

```
<div class="container"> <div class="row"> <div class="col align-self-start">Column one</div>  
<div class="col align-self-center">Column two</div>  
  <div class="col align-self-end">Column three</div> </div> </div>
```

Alinierea orizontală a coloanelor grilei

Puteți utiliza clasele **.justify-content-start**, **.justify-content-center**, și **.justify-content-end** pentru a alinia coloanele grilei pe orizontală la stânga, centrul și respectiv dreapta unui container. Să vedem cum funcționează următorul exemplu:

```
<div class="container"> <div class="row justify-content-start">  
  <div class="col-4">Column one</div>  
  <div class="col-4">Column two</div> </div>  
  <div class="row justify-content-center"> <div class="col-4">Column one</div>  
  <div class="col-4">Column two</div> </div>  
  <div class="row justify-content-end"> <div class="col-4">Column one</div>  
  <div class="col-4">Column two</div> </div> </div>
```

Alternativ, puteți utiliza clasa **.justify-content-around** pentru a distribui uniform coloanele grilei cu spații de jumătate de dimensiune la fiecare capăt, în timp ce puteți utiliza clasa **.justify-content-between** pentru a distribui uniform coloanele grilei, unde prima coloană este plasată la început și ultima coloană la sfârșit. Încercați următorul exemplu pentru a vedea cum funcționează de fapt:

```
<div class="container"> <div class="row justify-content-around"> <div class="col-4">Column one</div>
<div class="col-4">Column two</div> </div>
<div class="row justify-content-between"> <div class="col-4">Column one</div>
<div class="col-4">Column two</div> </div> </div>
```

Reordonarea coloanelor grilei

Puteți chiar schimba ordinea vizuală a coloanelor grilei fără a le schimba ordinea în markup-ul propriu-zis. Folosiți clasa **.order-last** pentru a ordona coloana la ultimul loc, în timp ce folosiți clasa **.order-first** pentru a ordona coloana la primul loc. Să vedem un exemplu:

```
<div class="container"> <div class="row"> <div class="col order-last">First, but ordered at last</div>
<div class="col">Second, but unordered</div>
<div class="col order-first">Last, but ordered at first</div>
</div> </div>
```

De asemenea, puteți utiliza **.order-*** clasele pentru a ordona coloanele grilei în funcție de numerele de ordine. Coloana grilei cu număr de ordine mai mare vine după coloana grilei cu număr de ordine mai mic sau coloana grilei fără clase de ordine. Include suport pentru clasele de la 1 la 12 pe toate cele cinci niveluri ale grilei.

```
<div class="container"> <div class="row"> <div class="col order-4">First,  
but ordered at last</div>  
<div class="col">Second, but ordered at first</div>  
<div class="col order-1">Last, but ordered at second</div> </div> </div>
```

Decalarea coloanelor grid-ului

De asemenea, puteți muta coloanele grid-ului la dreapta în scopul alinierii utilizând clasele de offset al coloanelor, cum ar fi **.offset-sm-***, **.offset-md-***, **.offset-lg-*** și așa mai departe.

Aceste clase compensează coloanele prin simpla creștere a marginii lor stânga cu numărul specificat de coloane. De exemplu, clasa **.offset-md-4** on column **.col-md-8** mută la dreapta cu patru coloane față de poziția sa inițială. Încercați următorul exemplu pentru a vedea cum funcționează:

```
<div class="container"> <div class="row"> <div class="col-sm-4"></div>  
<div class="col-sm-8"></div> </div>  
<div class="row"> <div class="col-sm-8 offset-sm-4"><!--Column with 4  
columns offset--></div> </div> </div>
```

De asemenea, puteți decala coloanele folosind clasele utilitare margin. Aceste clase sunt utile în situațiile în care lățimea decalajului nu este fixă. Iată un exemplu:

```
<div class="container"> <div class="row"> <div class="col-md-4"></div>  
<div class="col-md-4 ms-auto"><!--Offset this column to right--></div> </div>  
<div class="row"> <div class="col-auto me-auto"></div>  
<div class="col-auto"><!--Move this column away from previous column--></div>  
</div> </div>
```

Crearea coloanelor compacte

Puteți elimina marginile implicite dintre coloane pentru a crea machete compacte adăugând clasa **.g-0** on **.row**. Această clasă elimină marginile negative de pe rând și padding-ul orizontal din toate coloanele secundare imediate. Iată un exemplu:

```
<div class="row g-0"> <div class="col-4">Column one</div>  
<div class="col-4">Column two</div>  
<div class="col-4">Column three</div> </div>
```

Separarea coloanelor la o linie nouă

De asemenea, puteți crea coloane cu lățime egală care se întind pe mai multe rânduri inserând o clasă `<div>`with` .w-100` acolo unde doriți ca coloanele să se întrerupă la o linie nouă. În plus, puteți face ca aceste întreruperi să fie responsive combinând clasa **.w-100** cu clase utilitare de afișare responsive.

```
<div class="container"> <!-- Break columns on all devices -->
<div class="row"> <div class="col">.col</div>
<div class="col">.col</div> <div class="w-100"></div>
<div class="col">.col</div>
<div class="col">.col</div> </div>
<!-- Break columns on all devices except extra large devices -->
<div class="row"> <div class="col">.col</div>
<div class="col">.col</div>
<div class="w-100 d-xl-none"></div>
<div class="col">.col</div>
<div class="col">.col</div> </div> </div>
```